

# Senior Mobile Developer Assessment: Route & Car Navigation

**Timebox:** We recommend about 3 days of work. The hard limit is 5 calendar days from when you receive this brief. Completing any items in section 5 (Bonus) earns extra marks, but they are optional and you should finish the required features first.

**Platform:** Flutter. Android is required. iOS is optional (see section 5), since not every candidate has access to a Mac.

**Location** must be implemented natively, as described in section 3.1.

**Cost:** This task must be completed using free, keyless services only. You do not need a credit card or a billing account for anything.

## 1. The Task

Build a single-screen Flutter app that:

- Gets the user's current location through your own native implementation.
- Fetches a driving route from that location to a destination the user selects by long-pressing on the map.
- Animates a car marker along that route, similar to the live tracking screen of a ride-hailing app.

## 2. Required Stack (Free Services Only)

| Need                         | Use                                                    |
|------------------------------|--------------------------------------------------------|
| Map                          | flutter_map with OpenStreetMap tiles                   |
| Routing                      | Public OSRM API                                        |
| Location (Android, required) | Your own Kotlin code using FusedLocationProviderClient |



Location (iOS, optional)      Your own Swift code using CoreLocation

Flutter ↔ native bridge      MethodChannel / EventChannel

OSRM endpoint example:

<https://router.project-osrm.org/route/v1/driving/{startLng},{startLat};{endLng},{endLat}?overview=full&geometries=polyline>

Note that OSRM expects coordinates in longitude, latitude order.

## Rules

- Do not use any third-party location or permission plugins (for example geolocator, location, or permission\_handler). Location permission handling and location fetching must be written by you in native code and exposed through your own platform channels. Submissions that use such a plugin will score zero for the native location section (3.1).
- Do not use any service that requires a billing account or API key (for example, Google Directions / Routes API, Mapbox).
- When using OpenStreetMap tiles, follow the OSM tile usage policy:
  - set a proper user-agent / package name (for example, `userAgentPackageName` in `flutter_map`), and
  - display visible attribution on the map ("© OpenStreetMap contributors").
- The public OSRM server is a shared demo service with a limit of roughly one request per second and is not intended for heavy use. Do not send a route request on every gesture. Debounce or throttle requests so rapid long-presses do not flood the server.

## 3. Functional Requirements

### 3.1 Location (Native)

#### Native implementation

- Implement location permission requests and location fetching in Kotlin (Android), exposed to Flutter through a platform channel. A Swift (iOS) implementation is optional.
- Provide two capabilities to Dart:
  - a one-shot current location call (for example, `getCurrentLocation()` returning a Future)
  - a continuous location stream (for example, via EventChannel) for live updates



- When Dart cancels the stream, the native side must stop requesting location updates. No updates may continue running in the background after the stream is cancelled or the screen is closed.
- Native errors must reach Dart as clear, typed errors (for example, permission denied, services disabled, timeout), not generic exceptions or raw strings that the UI has to parse.
- On the Dart side, hide the channel behind a clean interface (for example, a `LocationService` class), so the rest of the app never touches the channel directly.
- Design the channel contract so that an iOS implementation could be added later without changing any Dart code. If you do not implement iOS, the Dart side should report a clear "not supported" error on that platform rather than crashing.

## Behaviour

- Request location permission at an appropriate moment, not immediately on app launch before the user has any context.
- Handle every permission state gracefully:
  - granted
  - denied
  - permanently denied (offer a way to open the app's Settings page, implemented natively)
  - location services turned off on the device
- Show a clear loading state while the first location fix is acquired, and handle the case where no fix arrives within a reasonable time.

## 3.2 Route

- Fetch the route from the user's location to the selected destination.
- Decode the polyline and draw it on the map.
- Fit the camera to show the whole route once it loads.
- Display total distance and estimated time.
- Handle errors clearly: no route found, network failure, and slow responses.
- If the user selects a new destination while a request is still in progress, the old request must not overwrite the newer result.
- Decide what happens if no location is available, for example because permission is denied, no fix arrives, or services are off. You may block routing, let the user choose a start point manually, or use another approach, but the app must not get stuck or crash. Explain your choice in DECISIONS.md.

## 3.3 Car Navigation

- A car icon moves along the route smoothly, not jumping from point to point.
- The car's speed should be roughly constant, regardless of how densely or sparsely the route points are spaced.



- The car rotates to face its direction of travel. Rotation must take the shortest path (for example, turning from 359° to 1° must not spin the long way round).
- The camera follows the car. If the user pans the map manually, following stops and a Recenter button appears.
- Controls: Start, Pause / Resume, Reset, and a speed multiplier (1x / 2x / 5x).
- Show remaining distance and remaining time, updating live as the car moves.
- The app must handle being sent to the background and brought back during animation without crashing, freezing, or leaking timers.

## 4. Senior-Level Requirements

- **Architecture:** Keep location, routing, and animation logic clearly separated. Use the state management approach of your choice (Riverpod, Bloc, etc.) and explain why you chose it. The route-animation logic should be testable without a map widget.
- **Resource safety:** No UI updates after a screen or controller is disposed. Animation controllers, timers, streams, and native location listeners must be cleaned up correctly.
- **Build flavors:** Set up two build flavors, dev and prod, on Android:
  - Each flavor has its own app name (for example, "NavTest Dev" and "NavTest") and its own application ID, so both can be installed on the same device side by side.
  - The routing base URL must come from the flavor's configuration, not be hard-coded in the app logic. (Both flavors may point to the public OSRM server, but switching one to a different server must require only a config change.)
  - The dev build shows a visible indicator (for example, a "DEV" banner or badge) so it can't be mistaken for prod.
  - Use `flutter run --flavor` with Android productFlavors.
- **Robustness:** Real-world route data can be messy. Your implementation must not crash, stutter, or produce invalid values (NaN, infinite) when the route contains closely spaced or repeated points.

## 5. Bonus (Optional)

These are not required, but they will be considered:

- **iOS support:** Implement the native location layer in Swift using CoreLocation, matching the Android behaviour and channel contract, and set up the dev / prod flavors on iOS using schemes and build configurations. If you do this, also handle the iOS "approximate location" (reduced accuracy) setting sensibly.
- **Live GPS mode:** Add an option for the car to follow the device's real location (using your native location stream) instead of the simulated animation.
- **Off-route detection:** In live GPS mode, if the device moves more than about 50 metres from the route, fetch a new route automatically.



- **Snap to route:** In live GPS mode, when the position is slightly off the line, display the car on the nearest point of the route.
- **Navigation camera:** The camera tilts and rotates with the car's direction of travel.

## 6. Deliverables

- GitHub repository with a clean, logical commit history (not a single large commit).
- **README.md** covering:
  - how to build and run each flavor on Android (and on iOS, if implemented)
  - Flutter version and package versions used
  - known limitations
- **DECISIONS.md** (1–2 pages) covering:
  - why you chose your architecture and state management
  - how you designed the Flutter ↔ native location bridge (channel choice, error mapping, stream lifecycle)
  - how your interpolation and bearing logic works
  - how you structured the flavor configuration
  - what you would change before shipping this to production (battery usage, background location, routing server choice, cost at scale)
  - what you deliberately left out due to time
- **Screen recording** (1–2 minutes) of the app running on a real Android device, showing the permission flow, route selection, car animation, and both flavors installed side by side. If you implemented iOS, include a short clip from an iPhone or the iOS Simulator as well.

## 7. Next Step

Shortlisted candidates will be invited to a 30-minute live session to walk through their code.

## 8. Submission

Send the repository link and screen recording within 5 calendar days of receiving this brief, or by the deadline given in your invitation email if that is earlier. Make sure the repository is public, or grant access to the reviewer account provided.

If anything in this brief is unclear, make a reasonable assumption and note it in your README. Handling ambiguity sensibly is part of the assessment.

Good luck!

